

Chapitre 2:

Cycle de vie du développement logiciel

2.1. Introduction au cycle de vie logiciel

Cycle de vie du logiciel

- Le **cycle de vie du logiciel** représente l'ensemble des **étapes et activités** nécessaires pour développer, mettre en service, maintenir et faire évoluer un logiciel, depuis l'expression du besoin jusqu'à son retrait.
- Le cycle de vie décrit **comment un logiciel évolue tout au long de son existence.**

Un logiciel a une vie

- Comme tout produit, un logiciel passe par différentes étapes :
- Durant sa vie, le logiciel peut être: développé, testé, déployé, utilisé, corrigé, amélioré, adapté à de nouveaux besoins, et éventuellement remplacé.
- Le cycle de vie couvre toute l'existence du logiciel, et pas uniquement sa phase de programmation.



Le cycle de vie organise le développement

- Le cycle de vie permet de définir :
 - **quoi faire**
 - **dans quel ordre** réaliser les activités
 - **quels résultats** produire
 - **qui intervient** dans chaque activité
 - **quand vérifier** la qualité du logiciel.
- Exemple:

Phase	Activité principale	Résultat
Besoins	Comprendre les attentes	Spécification des besoins
Analyse	Comprendre le problème	Modèle d'analyse
Conception	Définir la solution	Modèle de conception
Implémentation	Programmer	Code source
Tests	Vérifier	Système validé
Déploiement	Mettre en production	Système opérationnel
Maintenance	Corriger / améliorer	Nouvelles versions

Pourquoi structurer le développement logiciel ?

- Pourquoi ne pas simplement commencer à programmer ?
 - Pour un petit programme, on peut parfois :
 - Problème → Code → Résultat
 - Mais pour un système complexe :
 - Besoins → Analyse → Conception → Code → Tests → Déploiement → Maintenance
- Une démarche structurée permet de :
 - mieux comprendre les besoins
 - réduire les erreurs
 - maîtriser la complexité
 - organiser le travail de l'équipe
 - faciliter la communication
 - contrôler l'avancement
 - améliorer la qualité du logiciel.

**Plus le système est complexe,
plus une démarche structurée
devient nécessaire.**

Que se passe-t-il sans démarche structurée ?

- Une mauvaise organisation peut entraîner :

- des besoins mal compris
- des fonctionnalités inutiles ou manquantes
- des erreurs de conception
- du code difficile à maintenir
- des retards
- une augmentation des coûts
- des difficultés de test
- une insatisfaction des utilisateurs.

**Une erreur détectée tardivement
coûte généralement plus cher à
corriger.**

- **Exemple**

- Un client demande : « Je veux pouvoir annuler une commande. »
 - Si cette exigence n'est pas correctement analysée et prise en compte dans la conception, l'ajout de cette fonctionnalité **après le développement** peut nécessiter des modifications importantes du système.

Les bénéfices d'un cycle de vie structuré

- Une démarche structurée permet de mieux maîtriser le projet



Les principales phases d'un développement logiciel

- Les principales phases sont :
 - Analyse des besoins
 - Analyse du système
 - Conception
 - Implémentation
 - Tests et validation
 - Déploiement
 - Maintenance et évolution
- Ces phases sont communes à de nombreuses démarches de développement, mais **leur organisation et leur enchaînement dépendent du modèle de cycle de vie choisi.**

Une démarche qui n'est pas nécessairement linéaire

- Le développement logiciel est rarement parfaitement linéaire
 - Même si les phases sont présentées dans un ordre logique :
 - Besoins → Analyse → Conception → Implémentation → Tests
 - la réalité implique souvent des **retours en arrière**
- Par exemple :
 - un test peut révéler une erreur de conception
 - la conception peut révéler un besoin incomplet
 - un changement de besoin peut nécessiter une modification du code.
- **Les phases du cycle de vie sont conceptuellement distinctes, mais elles peuvent interagir et se répéter.**
- Cette idée sera essentielle lorsque nous aborderons les modèles **itératif, incrémental, en spirale et agile.**

Le cycle de vie logiciel

- Le cycle de vie logiciel
 - décrit les étapes de l'existence d'un logiciel
 - structure les activités de développement
 - facilite l'organisation du projet
 - contribue à maîtriser la complexité, les coûts et les risques
 - favorise la qualité et la maintenabilité
 - accompagne le logiciel jusqu'à sa maintenance et son évolution.
- Les grandes phases

Besoins → Analyse → Conception → Implémentation → Tests → Déploiement → Maintenance
- Les phases sont-elles toujours réalisées de la même manière et dans le même ordre ?

2.2. Modèles de cycle de vie

Qu'est-ce qu'un modèle de cycle de vie ?

- Un modèle de cycle de vie définit la manière dont les différentes phases du développement logiciel sont organisées, enchaînées et exécutées.
- Les phases restent globalement les mêmes:
Besoins → Analyse → Conception → Implémentation → Tests → Déploiement → Maintenance
- Mais leur organisation peut varier.
- Un modèle de cycle de vie c'est comment organiser les différentes phases du développement logiciel ?

Pourquoi existe-t-il plusieurs modèles ?

- Les projets logiciels diffèrent par:
 - la stabilité des besoins
 - la complexité du système
 - le niveau de risque
 - les contraintes de coût et de délai
 - les exigences de qualité et de sécurité
 - le niveau d'implication du client
 - la nécessité de livrer rapidement
- Conséquence :
 - Il n'existe pas un modèle de cycle de vie adapté à tous les projets.

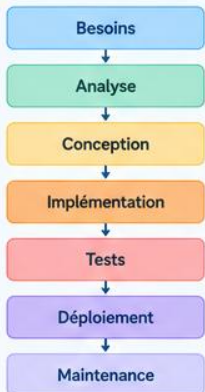
Principaux modèles de cycle de vie



Principaux modèles de cycle de vie

Les mêmes phases, mais une organisation différente !

1 Modèle en cascade (Waterfall)



Principe

Succession linéaire des phases, une après l'autre.



Atouts

- Simple et facile à comprendre
- Phases bien définies
- Bonne planification



Limites

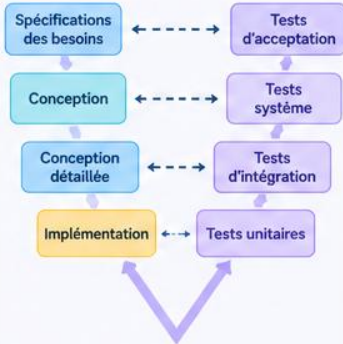
- Peu flexible aux changements
- Validation tardive
- Retour en arrière coûteux



Quand l'utiliser ?

Lorsque les besoins sont stables et bien définis.

2 Modèle en V



Principe

Associe chaque phase de développement à un niveau de test correspondant.



Atouts

- Forte importance des tests
- Tracabilité besoins ↔ tests
- Adapté aux environnements réglementés



Limites

- Peu flexible aux changements
- Retour en arrière coûteux
- Livraison en fin de processus



Quand l'utiliser ?

Lorsque la qualité et la validation sont essentielles.

3 Modèle itératif et incrémental



Principe

Développement progressif par itérations et ajout incrémental de fonctionnalités.



Atouts

- Feedback régulier
- Détection précoce des problèmes
- Adaptation aux changements



Limites

- Planification des itérations complexe
- Architecture globale difficile à maîtriser
- Risque de dette technique



Quand l'utiliser ?

Lorsque les besoins évoluent et qu'il est utile de livrer rapidement des versions successives.

4 Modèle en spirale



Principe

Approche itérative avec une attention particulière à la gestion des risques.



Atouts

- Gestion explicite des risques
- Adaptation progressive
- Adapté aux projets complexes



Limites

- Plus complexe à gérer
- Coût potentiellement élevé
- Nécessite une expertise en risques



Quand l'utiliser ?

Pour des projets complexes, coûteux ou à forts risques.

5 Modèles agiles



Principe

Livraisons fréquentes, adaptation aux changements et feedback continu.



Atouts

- Forte adaptation
- Livraisons régulières
- Implication du client
- Détection précoce des problèmes



Limites

- Forte collaboration nécessaire
- Implication régulière du client
- Moins adapté aux exigences figées



Quand l'utiliser ?

Lorsque les besoins sont évolutifs et qu'il faut une grande flexibilité.



Les modèles de cycle de vie ne changent pas nécessairement ce qu'il faut faire, mais principalement la manière d'organiser et d'enchaîner les activités.

Modèle en cascade

Waterfall

Modèle en cascade (Waterfall)

- Le modèle en cascade organise le développement sous forme d'une **succession linéaire de phases**.



Modèle en cascade (Waterfall)

- Chaque phase est principalement réalisée avant de passer à la suivante.
- On avance progressivement d'une phase à l'autre, comme une cascade.
- Quand l'utiliser ?
 - Lorsque les besoins sont bien définis et relativement stables et que le projet est fortement prévisible.

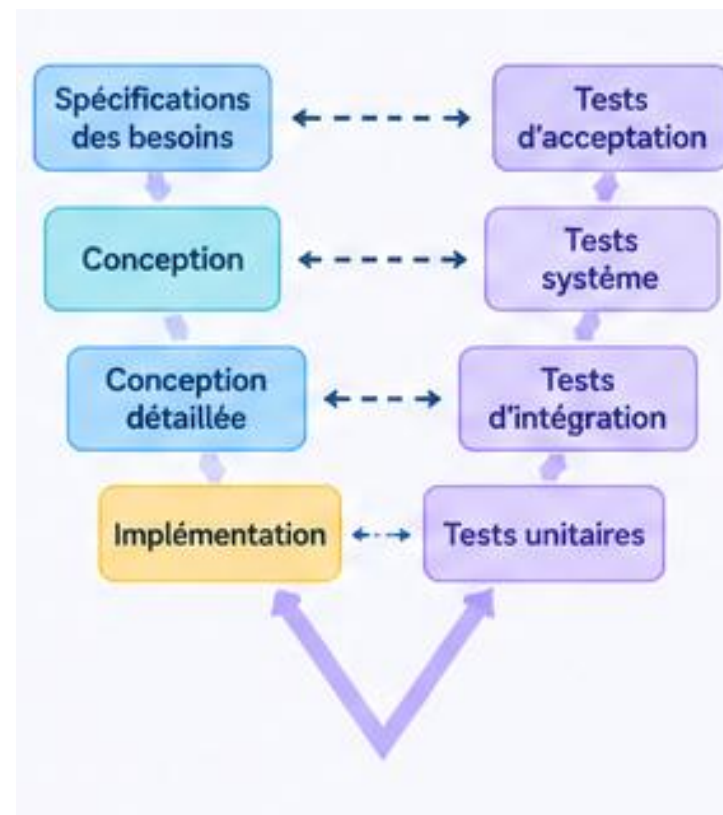
Avantages et limites du modèle en cascade

- Avantages :
 - processus simple et facile à comprendre
 - phases clairement définies
 - planification et documentation importantes
 - progression facilement identifiable
 - adapté aux projets dont les besoins sont stables
- Limites :
 - faible capacité d'adaptation aux changements
 - validation souvent tardive
 - retour en arrière coûteux
 - problèmes parfois détectés tardivement
 - produit fonctionnel disponible relativement tard

Modèle en V

Modèle en V

- Le modèle en V est une évolution du modèle en cascade qui met particulièrement l'accent sur la vérification et la validation.
- Les phases de développement sont associées à des niveaux de tests correspondants.



Modèle en V: correspondance développement / tests

Développement	Validation associée
Spécifications des besoins	Tests d'acceptation
Conception	Tests système
Conception détaillée	Tests d'intégration
Implémentation	Tests unitaires

- Chaque étape de développement prépare le niveau de test qui permettra de vérifier le résultat obtenu.
- Le modèle en V cherche donc à intégrer la préparation des tests dès les premières phases du projet.

Avantages et limites du modèle en V

- Avantages
 - forte importance accordée aux tests
 - traçabilité entre besoins et tests
 - processus structuré
 - adapté aux environnements fortement réglementés
 - documentation importante
- Limites
 - faible flexibilité face aux changements
 - retour en arrière coûteux
 - nécessite des besoins relativement stables
 - livraison du système principalement en fin de processus
- Le modèle en V met l'accent sur la qualité et la validation, mais reste principalement séquentiel.

Modèle itératif et incrémental

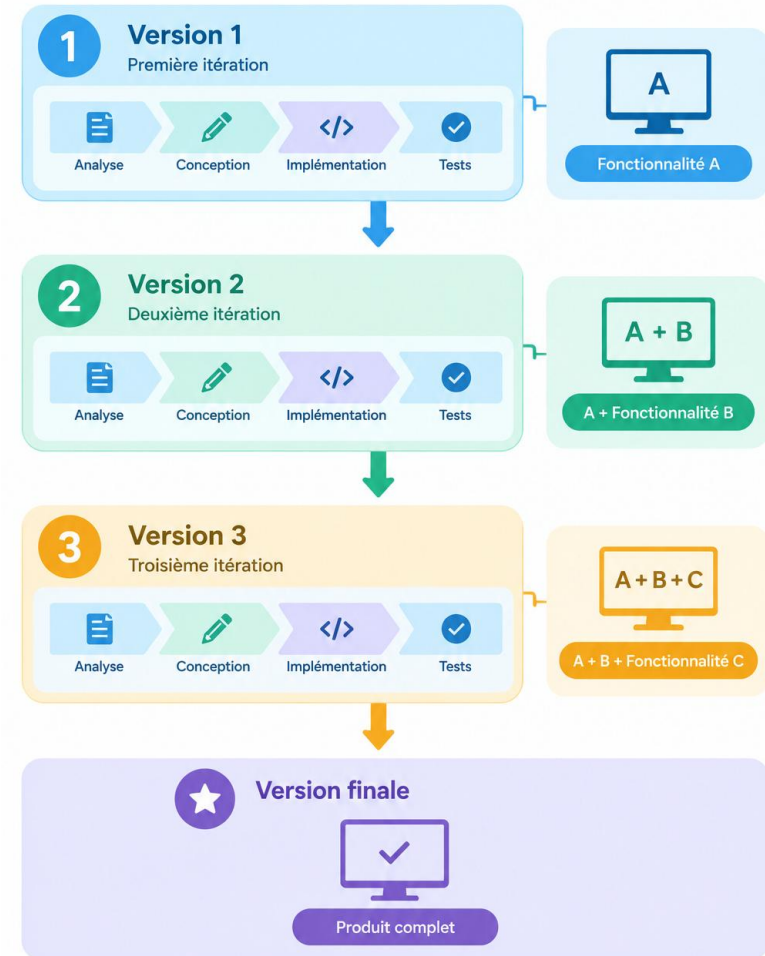
Modèle itératif et incrémental: principe

- Le système est développé progressivement à travers plusieurs itérations.
- Chaque itération permet d'améliorer ou d'étendre le système.
- Chaque itération peut comporter :
Analyse → Conception → Implémentation → Tests



Itération et incrément

- Il est important de distinguer les deux notions.
 - Itératif :
 - On répète le processus pour améliorer progressivement le système.
 - Incrémental :
 - On ajoute progressivement de nouvelles fonctionnalités au système.
- Itératif : améliorer
- Incrémental : ajouter
- Un projet peut être à la fois itératif et incrémental.



Avantages et limites du modèle itératif et incrémental

- Avantages
 - feedback régulier
 - détection plus précoce des problèmes
 - livraison progressive
 - adaptation plus facile aux changements
 - réduction du risque
- Limites
 - nécessite une bonne planification des itérations
 - architecture globale parfois difficile à maîtriser
 - risque de dette technique
 - intégration et cohérence doivent être surveillées
- Quand l'utiliser ?
 - Lorsque les besoins sont susceptibles d'évoluer et qu'il est utile de disposer rapidement de versions successives du logiciel.

Modèle en spirale

Principe du modèle en spirale

- Le modèle en spirale combine une approche itérative avec une attention particulière portée à la gestion des risques.
- Chaque boucle de la spirale représente une itération.
- Chaque itération permet de réduire progressivement les risques du projet.



Une approche centrée sur les risques

- Chaque cycle comprend généralement les quatre activités de la spirale:
 1. Définir les objectifs
 2. Identifier et analyser les risques
 3. Développer et tester
 4. Evaluer les résultats et planifier la prochaine itération
- Le processus se répète jusqu'à obtenir une solution satisfaisante.

Avantages et limites du modèle en spirale

- Avantages

- gestion explicite des risques
- adaptation progressive
- feedback régulier
- adapté aux projets complexes
- permet d'explorer différentes solutions

- Limites

- plus complexe à gérer
- coût potentiellement élevé
- nécessite une expertise en gestion des risques
- moins adapté aux petits projets simples

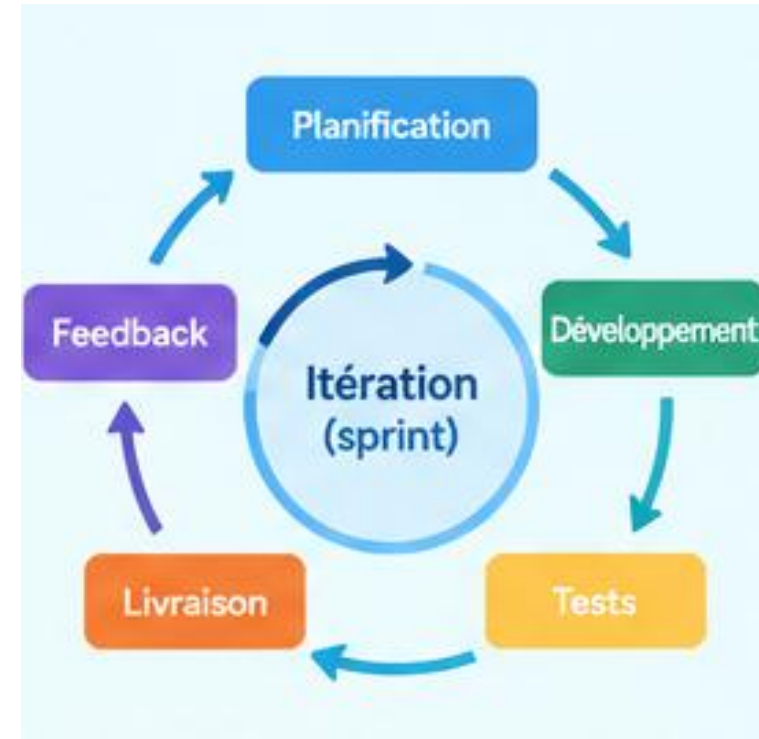
- Quand l'utiliser ?

- Principalement pour des projets complexes, coûteux ou présentant des risques importants.

Modèles Agiles

Les modèles agiles: principe général

- Les approches agiles privilégient:
 - les livraisons fréquentes
 - les interactions avec le client
 - l'adaptation aux changements
 - le développement itératif et incrémental
 - le feedback continu
- **Idée générale:**
 - construire progressivement le produit logiciel et s'adapter continuellement aux besoins.



Agile : une famille d'approches

- L'agilité n'est pas un modèle unique
 - Le terme Agile désigne une famille d'approches et de méthodes, notamment :
 - Scrum
 - Extreme Programming (XP)
 - Kanban
 - Lean Software Development
-
- Agile n'est pas synonyme de Scrum.
 - Scrum est une méthode agile parmi d'autres.

Avantages et limites des approches agiles

- Avantages

- forte capacité d'adaptation
- feedback fréquent
- livraisons régulières
- implication importante du client
- détection précoce des problèmes
- visibilité régulière sur l'avancement

- Limites

- nécessite une forte collaboration
- nécessite une implication régulière du client
- moins adapté lorsque les exigences sont strictement fixées
- documentation parfois moins importante selon l'approche
- nécessite une équipe autonome et organisée

Synthèse: différentes façons d'organiser le cycle de vie

- **Cascade** : Séquentiel
 - **Modèle en V** : Séquentiel + validation
 - **Itératif / Incrémental** : Développement progressif
 - **Modèle en spirale** : Itératif + gestion des risques
 - **Agile** : Itératif + incrémental + adaptation + feedback
-
- Les modèles de cycle de vie ne changent pas nécessairement ce qu'il faut faire, mais principalement la manière d'organiser et d'enchaîner les activités.